

SINTASSI DEI TEMPLATE IN VUE.JS



INTERPOLAZIONE DEL TESTO

l'interpolazione del testo viene effettuata tramite la sintassi "mustache" (doppie parentesi graffe).

```
<span>Message: {{ msg }}</span>
```

se msg contiene "hello world!", l'output html sarà:

```
<span>Message: Hello world!</span>
```

sicuro: l'html viene automaticamente sottoposto a escape. es., se msg è hello:

```
<span>Message: &lt;b&gt; Hello</span>
```



ESPRESSIONI JAVASCRIPT

puoi usare espressioni javascript:

```
{{ number + 1 }}
```

```
{{ ok ? 'YES' : 'NO' }}
```

```
{{ message.split('').reverse().join('') }}
```



ESPRESSIONI JAVASCRIPT (LIMITAZIONI)

non puoi usare controllo di flusso o dichiarazioni (statement):

```
<!-- non funzionerà -->
{{ var a = 1 }}

<!-- non funzionerà -->
{{ if (ok) { return message } }}
```



ESPRESSIONI JAVASCRIPT (FUNZIONI)

chiamare una funzione è permesso:

```
{{ formatDate(date) }}
```

nota: vue.js chiamerà queste funzioni ad ogni aggiornamento.



DIRETTIVE VUE.JS

nei template, vue.js aggiunge attributi personalizzati ai tag html.

questi attributi sono chiamati direttive.

struttura: `v-on:submit.prevent="onSubmit"`

- **nome:** inizia con `v-`. può essere omesso quando si usano le scorciatoie (shorthands).
- **argomento:** segue i due punti o il simbolo della scorciatoia.
- **modificatori:** denotati dal punto iniziale.
- **valore:** interpretato come espressioni javascript.



DIRETTIVE INTEGRATE IN VUE.JS

- `v-text`: imposta il contenuto testuale (sicuro: html auto-escaped).
- `v-html`: imposta il contenuto html (non sicuro, usa `v-text` invece).
- `v-show`: commuta la visibilità dell'elemento.
- `v-if`, `v-else-if`, e `v-else`: commuta il rendering dell'elemento.
- `v-for`: ripete l'elemento corrente (e il sottoalbero).
- `v-on`: registra un event listener.
- `v-bind`: lega un attributo a una variabile.
- `v-model`: binding bidirezionale (two-way) con una variabile.

direttive meno comuni: <https://vuejs.org/api/built-in-directives.html>



CONTENUTO DELL'ELEMENTO

imposta il contenuto testuale dell'elemento al valore di una variabile.

```
<span v-text="msg"></span>
```

se `msg` viene aggiornato, `vue.js` aggiornerà il contenuto dello `<span>` di conseguenza.



EVENTI

esegui una funzione quando succede qualcosa (es. un click).

```
<span v-on:click="alert('Hello!')">World</span>
```

sugli elementi html, tutti gli eventi standard sono supportati. quando usato su sfc (single file components), puoi usare eventi sfc. Scorciatoia:

```
<span @click="alert('Hello!')">World</span>
```



BINDING DEGLI ATTRIBUTI

`v-bind` crea un legame da javascript all'attributo dell'elemento.

```
<span v-bind:id="dynamicId"></span>
```

in questo esempio, l'attributo `id` per questo `<span>` verrà impostato al valore della variabile javascript `dynamicId`. Scorciatoia:

```
<span :id="dynamicId"></span>
```



BINDING BIDIREZIONALE DEGLI ATTRIBUTI

`v-model` crea un binding bidirezionale.

```
<input type="text" v-model="name" ></input>
```

solo per: `<input>`, `<select>`, `<textarea>` e componenti `vue.js`.



V-BIND VS V-MODEL

- `v-bind` serve per mostrare dati da javascript all'utente.
- `v-model` serve per mostrare dati e lasciare che l'utente li modifichi.

`v-model` è simile a `v-bind` + `v-on` più del codice di collegamento (glue code).



	Mustache ({{ data }})	Direttive (v-bind / :)	Direttive (v-model)	Direttive (v-on / @)
Target DOM	Contenuto Testuale dell'elemento	Attributo dell'elemento (es. href, class, id, src)	Valore/Proprietà dell'input	Eventi dell'elemento (es. click, submit)
Anticassi	{{ ... }}	Inizia con v-bind: o :	Inizia con v-model:	Inizia con v-on: o @
Flusso dati	JS → DOM (Solo Output)	JS → DOM (Solo Output)	DOM ↔ JS (Input/Output)	DOM → JS (Input Evento)
Esempio	Ciao {{ name }}	<div :class="myClass"></div>	<input v-model="name">	<button @click="save">



RENDERING CONDIZIONALE

per renderizzare condizionalmente una parte del template, usa `v-if`:

```
<p v-if="showMessage">  
  Vue.js renderizzerà questa parte solo quando showMessage è vero  
</p>
```



VISIBILITÀ CONDIZIONALE

per mostrare/nascondere una parte del template, usa `v-show`:

```
<p v-show="showMessage">  
  Questa parte è visibile solo quando showMessage è vero  
</p>
```

nota: l'elemento viene comunque renderizzato, ma è invisibile!



CICLI (LOOPING)

v-for ripeterà l'elemento (e il suo contenuto) per ogni iterazione.

```
<div v-for="item in items">{{ item }}</div>
```

in questo esempio, se `items` è l'array ['a', 'b'], l'html risultante è:

```
<div>a</div>  
<div>b</div>
```



LINK UTILI

- <https://vuejs.org/guide/essentials/template-syntax.html>
- <https://vuejs.org/api/built-in-directives.html>
- <https://developer.mozilla.org/en-US/docs/Web/Events>

